Practical 14: Database Integration with Backend Application

Aim

To study and implement database integration with a Python backend application using SQLite.

Theory

SQLite is a lightweight, serverless, self-contained, file-based (or in-memory) relational database engine that requires no separate database server process or configuration, making it ideal for embedded applications, prototypes, and testing; it is bundled with Python's standard library via the sqlite3 module, so no external installation is needed. It allows applications to create tables with defined schemas (column names and data types), insert, update, delete, and query records using standard SQL (Structured Query Language) statements. A Connection object, created via sqlite3.connect(), represents an open session with the database (which can be a file path or the special value ':memory:' for a temporary, non-persistent in-memory database, as used in this practical), and is responsible for committing or rolling back transactions via conn.commit(). A Cursor object, obtained from the connection, is used to execute SQL commands (cur.execute()) and iterate over or fetch the results of queries (fetchone(), fetchall()), acting as a control structure over the result set. Backend database integration like this forms the persistence layer of most real-world applications, storing data that must survive beyond a single program execution.

Algorithm

1. Import the sqlite3 module and create an in-memory database connection.
2. Create a cursor object from the connection.
3. Execute a CREATE TABLE statement to define the students table schema.
4. Execute INSERT statements to add student records and commit the transaction.
5. Execute a SELECT statement to retrieve all records and print them.
6. Close the database connection.

Program

```
import sqlite3

conn = sqlite3.connect(":memory:")
cur = conn.cursor()

cur.execute("""
    CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)
""")

cur.execute("INSERT INTO students (name, marks) VALUES ('Amit', 85)")
cur.execute("INSERT INTO students (name, marks) VALUES ('Priya', 92)")
conn.commit()

cur.execute("SELECT * FROM students")
print("Database Records:")
for row in cur.fetchall():
    print(row)

conn.close()
```

Output

```
Database Records:
(1, 'Amit', 85)
(2, 'Priya', 92)
```

Result / Conclusion

Database integration with a Python backend was successfully implemented using SQLite, with table creation, insertion, and retrieval of records verified.

Viva Questions

Q1. What is the difference between SQLite and MySQL/PostgreSQL?

Ans: SQLite is serverless, file-based (or in-memory), zero-configuration, and single-process — ideal for embedded use, mobile apps, and prototyping. MySQL and PostgreSQL are client-server databases requiring a separate running server process, supporting multiple concurrent connections, user authentication, and large-scale production workloads with better performance, concurrency control (MVCC in PostgreSQL), and advanced features. SQLite is not suitable for high-concurrency web applications.

Q2. What is the purpose of the cursor in sqlite3?

Ans: A cursor (cur = conn.cursor()) is an interface to execute SQL commands and fetch results. cur.execute(sql) sends a query to the database engine. cur.fetchone() returns the next row as a tuple (or None). cur.fetchall() returns all remaining rows as a list of tuples. The cursor maintains the current position in the result set, allowing row-by-row iteration. A connection can have multiple cursors for parallel queries.

Q3. Why is conn.commit() necessary after INSERT/UPDATE/DELETE?

Ans: SQLite (and most relational databases) use transactions — a group of statements treated as an atomic unit. Changes made by INSERT/UPDATE/DELETE are held in a pending transaction in memory until conn.commit() makes them permanent (durable) in the database file. Without commit(), the changes are lost when the connection closes. conn.rollback() discards the pending changes. SELECT queries do not need commit() as they do not modify data.

Q4. What is the difference between ':memory:' and a file-path database?

Ans: sqlite3.connect(':memory:') creates a temporary database entirely in RAM — extremely fast, but all data is lost when the connection closes or the program exits. It is ideal for testing and temporary computation. sqlite3.connect('mydb.sqlite') creates or opens a persistent database file on disk — data survives program restarts. For production applications, always use a file-based database.

Q5. What is SQL injection and how can it be prevented in sqlite3?

Ans: SQL injection occurs when user-supplied input is concatenated directly into an SQL string, allowing an attacker to insert malicious SQL code. Example: f"SELECT * FROM users WHERE name='{name}'" — if name is "' OR '1'='1", it returns all rows. Prevention: always use parameterised queries with placeholders: cur.execute('SELECT * FROM users WHERE name=?', (name,)). The sqlite3 module safely escapes the value, preventing injection.